

Contents

The course will mainly cover the following topics:

- ✓ A Gentle Introduction to Machine Learning
- ✓ Linear Regression
- ✓ Logistic Regression
- ✓ Naive Bayes
- ✓ Support Vector Machines
- ✓ Decision Trees and Ensemble Learning
- ✓ Clustering Fundamentals
- ✓ Hierarchical Clustering
- ✓ Neural Networks and Deep Learning
- ✓ Unsupervised Learning

Gradient descent for multiple variables

Gradient Descent for Multiple Variables (Multivariable Gradient Descent)

Gradient Descent is an optimization algorithm used in Machine Learning to minimize the cost function (error).

When there are multiple input variables/features, one can use Multivariable

Gradient Descent.

It is widely used in:

- Linear Regression
- Logistic Regression
- Neural Networks
- Deep Learning

Linear Regression with Multiple Variables

Suppose we want to predict **house price** using:

The hypothesis (prediction equation) becomes:

- x_1 = house size
- x_2 = number of bedrooms

The hypothesis (prediction equation) becomes:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

Where:

- $h_{\theta}(x)$ = predicted value
- θ_0 = bias/intercept
- θ_1, θ_2 = parameters/weights

Cost Function

The cost function measures how wrong our predictions are.
For multiple variables:

$$J(\theta_0, \theta_1, \theta_2) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Where:

- m = number of training examples
- $y^{(i)}$ = actual value
- $h_{\theta}(x^{(i)})$ = predicted value

Goal:

- Minimize $J(\theta)$

Gradient Descent Update Rule

For multiple variables:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

Where:

- α = learning rate
- θ_j = parameter
- $\frac{\partial}{\partial \theta_j}$ = partial derivative



Update Formula for Each Parameter

For multivariable linear regression:

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

This update runs for:

- θ_0
- θ_1
- θ_2
- ...
- θ_n

Step-by-Step Example

1

Suppose we want to predict exam score using:

Hours Studied (x_1)	Sleep Hours (x_2)	Score (y)
1	6	50
2	5	55
3	8	70

So: $m = 3$

Hypothesis Function $\gg h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$

Assume initial values:

- $\theta_0 = 0$
 - $\theta_1 = 0$
 - $\theta_2 = 0$
- and Learning rate:
- $\alpha = 0.01$

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

- $\theta_0 = 0$
- $\theta_1 = 0$
- $\theta_2 = 0$

First Prediction

2

Since all theta values are 0: $h_{\theta}(x) = 0$

For all training examples:

- prediction = 0

Errors become:

Actual y	Prediction	Error
50	0	-50
55	0	-55
70	0	-70

Update θ_0

3

Formula:

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

Substitute values:

$$\theta_0 = 0 - 0.01 \times \frac{1}{3} (-50 - 55 - 70) = 0.583$$

Explanation

Update θ_0

Formula:

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

Substitute values:

$$\theta_0 = 0 - 0.01 \times \frac{1}{3}(-50 - 55 - 70) = 0.583$$



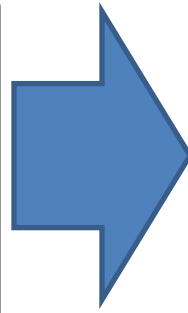
Example	y	Prediction $h_{\theta}(x)$	Error $h_{\theta}(x) - y$
1	50	0	$0 - 50 = -50$
2	55	0	$0 - 55 = -55$
3	70	0	$0 - 70 = -70$

Step 1: Sum All Errors	$\begin{aligned} &\sum (h_{\theta}(x) - y) \\ &= -50 + (-55) + (-70) \\ &= -175 \end{aligned}$
Step 2: Divide by m	$\begin{aligned} &\frac{1}{m} \sum (h_{\theta}(x) - y) \\ &= \frac{-175}{3} \\ &= -58.333 \end{aligned}$
Step 3: Multiply by Learning Rate	$\begin{aligned} &\alpha \times \left(\frac{-175}{3} \right) \\ &= 0.01 \times (-58.333) \\ &= -0.58333 \end{aligned}$
Step 4: Update θ_0	Formula: $\theta_0 = 0 - (-0.58333)$ Subtracting a negative means addition: $\theta_0 = 0 + 0.58333$ Final answer: $\theta_0 \approx 0.583$

Update θ_1

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})x_1^{(i)}$$

$$\theta_1 = 0 - 0.01 \times \frac{1}{3} [(-50)(1) + (-55)(2) + (-70)(3)]$$
$$= 1.233$$



Update θ_1

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})x_1^{(i)}$$

Example	x_1	y	Prediction	Error
1	1	50	0	-50
2	2	55	0	-55
3	3	70	0	-70

Multiply Error by x_1

$$(h_{\theta}(x) - y)x_1$$

1

Add Them Together $-50 + (-110) + (-210)$

$$= -370$$

Divide by m :

$$\frac{-370}{3}$$

$$= -123.333$$

Multiply by Learning Rate: $0.01 \times (-123.333)$

$$= -1.23333$$

Update θ_1

Initial value: $\theta_1 = 0$

Now apply formula: $\theta_1 = 0 - (-1.23333)$
 $= 1.23333$

3

Multiply Error by x_1

$$(h_{\theta}(x) - y)x_1$$

Example 1  $(-50)(1) = -50$

Example 2  $(-55)(2) = -110$

Example 3  $(-70)(3) = -210$

2

Update θ_2

$$\theta_2 := \theta_2 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_2^{(i)}$$

$$\theta_2 = 0 - 0.01 \times \frac{1}{3} [(-50)(6) + (-55)(5) + (-70)(8)] = 3.783$$

Explanation

Add All Values

$$-300 + (-275) + (-560) = -1135$$

Divide by m

$$\frac{-1135}{3} = -378.333$$

2

Multiply by Learning Rate

$$0.01 \times (-378.333) \\ = -3.78333$$

Update θ_2

$$\theta_2 = 0$$

$$\theta_2 = 0 - (-3.78333)$$

$$\theta_2 = 3.78333$$

3

Training Data

Example	x_2 (Sleep Hours)	Actual y	Prediction	Error $h_{\theta}(x) - y$
1	6	50	0	-50
2	5	55	0	-55
3	8	70	0	-70

Multiply Error by x_2 $(h_{\theta}(x) - y) x_2$

$$\text{Example 1} \Rightarrow (-50)(6) = -300$$

$$\text{Example 2} \Rightarrow (-55)(5) = -275$$

$$\text{Example 3} \Rightarrow (-70)(8) = -560$$

1

Updated Parameters

After 1 iteration:

Parameter	Value
θ_0	0.583
θ_1	1.233
θ_2	3.783

New hypothesis:

$$h_{\theta}(x) = 0.583 + 1.233x_1 + 3.783x_2$$

Now predictions improve.

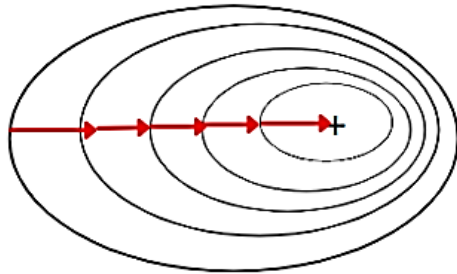
Gradient descent repeats this process many times until the cost becomes very small.

Different Types of Gradient Descent Algorithms

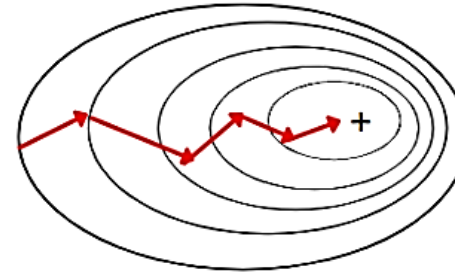
Gradient descent algorithm can be divided into three categories

1. **Batch Gradient Descent**
2. **Stochastic Gradient Descent (SGD)**
3. **Mini Batch Gradient Descent**

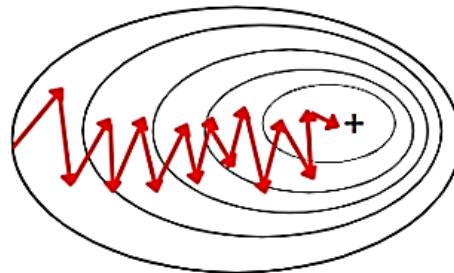
Batch Gradient Descent



Mini-Batch Gradient Descent



Stochastic Gradient Descent



Batch Gradient Descent

- ✓ **In batch gradient descent, the update of model parameters or weights is calculated using the entire training dataset at once.** This means that for each step in the training process, the algorithm calculates the gradient of the cost function for the entire dataset.
- ✓ This algorithm is often used when the **training dataset is relatively small** and can fit into memory comfortably.
- ✓ **For example, consider a linear regression model where you're predicting housing prices based on features like size and location. If your dataset is small, you could use batch gradient descent to train your model, updating the weights based on the error calculated from the entire dataset.**

Repeat until convergence

{

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

}

Batch Gradient Descent

- ✓ Despite its accuracy and consistency, batch gradient descent has several drawbacks.
 - **The batch gradient descent method typically requires the entire training dataset in memory**
 - Large datasets can result in very slow model updates or training speeds.
 - It can be slow and computationally expensive(require more computational power) when dealing with large datasets and complex models.
 - Additionally, it can get stuck in shallow local minima or saddle points, where the gradient is either very small or zero.
 - Furthermore, it does not permit online or incremental learning, meaning new data cannot be added to the training set without restarting the algorithm.

Let's understand how the Gradient descent can be derived from cost function:

1. Cost function

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

2. Compute the partial derivative of the cost function with respect to θ_j

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$$

3. Update θ_j using the gradient descent update rule:

$$\theta_j := \theta_j - \alpha \frac{\partial \theta_j}{\partial J}$$

Substituting the partial derivative, we get:

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$$

This update rule is applied simultaneously for all θ_j until convergence is reached. The intuition behind this process is that we update each parameter in the direction that reduces the cost function, and the size of the update is determined by the learning rate (α). The sum term represents the average contribution of each training example to the update. The process is repeated iteratively until the algorithm converges to parameter values that minimize the cost function.

Stochastic Gradient Descent (SGD)

- ✓ Stochastic gradient descent updates the model's parameters using **only one training example or a small batch of examples at a time**. This means the gradient and hence the **parameter update is calculated and applied after each example or small batch**, making the process more incremental.
- ✓ SGD is particularly **useful when dealing with large datasets** that cannot fit into memory. For example, the model parameters are updated incrementally as each image (or a small batch of images) is processed, enabling the **model to learn progressively without the need to load the entire dataset into memory**.
- ✓ **We update the values of parameters after each example of training set. It is relatively faster than other types but its accuracy is less.**

for i in range (m):

$$\theta_j = \theta_j - \alpha (\hat{y}^i - y^i) x_j^i$$

Stochastic Gradient Descent (SGD)

Advantages

- ✓ You can instantly see your model's performance and improvement rates with frequent updates.
- ✓ This variant of the steepest descent method is probably the easiest to understand and implement, especially for beginners.
- ✓ Increasing the frequency of model updates will allow you to learn more about some issues faster.
- ✓ The noisy update process allows the model to avoid local minima (e.g., premature convergence).
- ✓ Faster and require less computational power.
- ✓ Suitable for the larger dataset.

Disadvantages

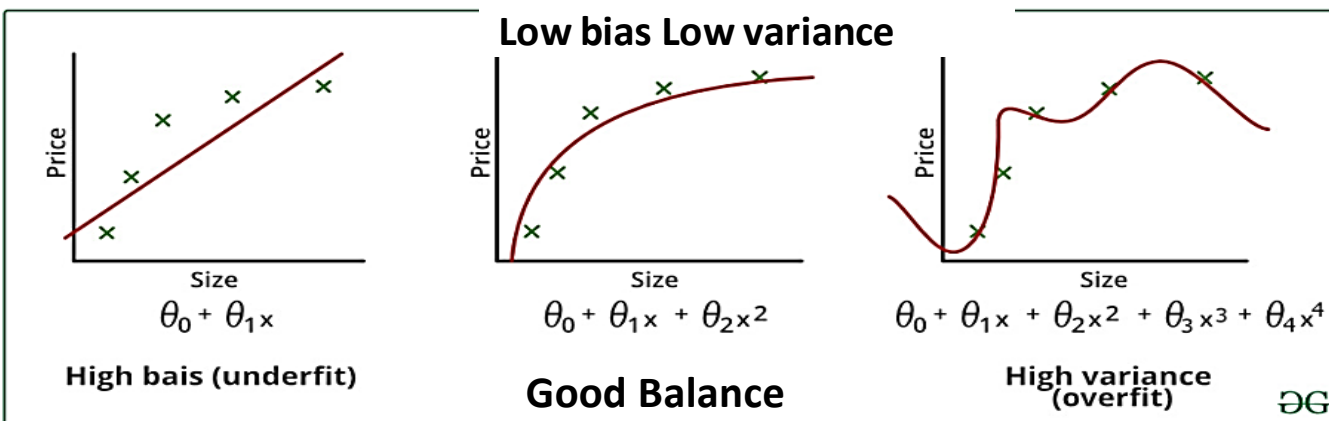
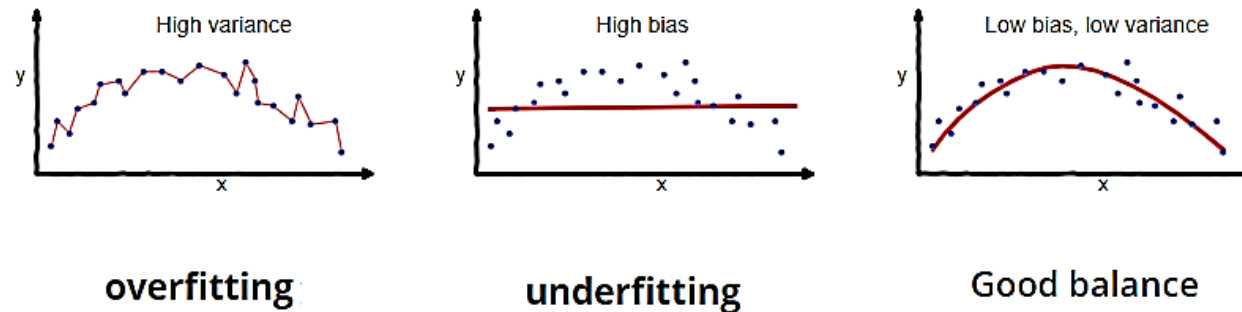
- ✓ Frequent model updates are more computationally intensive than other steepest descent configurations, and it takes considerable time to train the model with large datasets.
- ✓ Frequent updates can result in noisy gradient signals. This can result in model parameters and cause errors to fly around (more variance across the training epoch).
- ✓ A noisy learning process along the error gradient can also make it difficult for the algorithm to commit to the model's minimum error.

What is Bias?

- ✓ **Bias** refers to the errors which occur when we try to fit a statistical model on real-world data which does not fit perfectly well on some mathematical model. It may be defined by the differences or error occurring between the model's **predicted value and the actual value**.
- ✓ Let Y be the true value of a parameter, and let $\hat{Y}$ be an estimator of Y based on a sample of data. Then, the bias of the estimator $\hat{Y}$ is given by:
 - $\text{Bias}(\hat{Y}) = E(\hat{Y}) - Y$
- ✓ where $E(\hat{Y})$ is the expected value of the estimator $\hat{Y}$. It is the measurement of the model that how well it fits the data.
- ✓ The high-bias model will not be able to capture the dataset trend. It is considered as the underfitting model which has a high error rate. It is due to a very simplified algorithm.
- ✓ For example, a linear regression model may have a high bias if the data has a non-linear relationship.
- ✓ We can use a more complex model like Polynomial regression for non-linear datasets

Variance (“How sensitive the model is to small changes in training data.”)

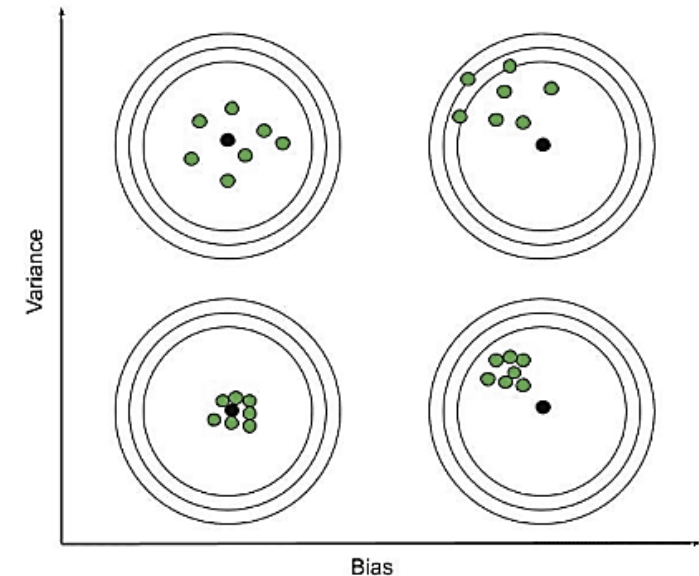
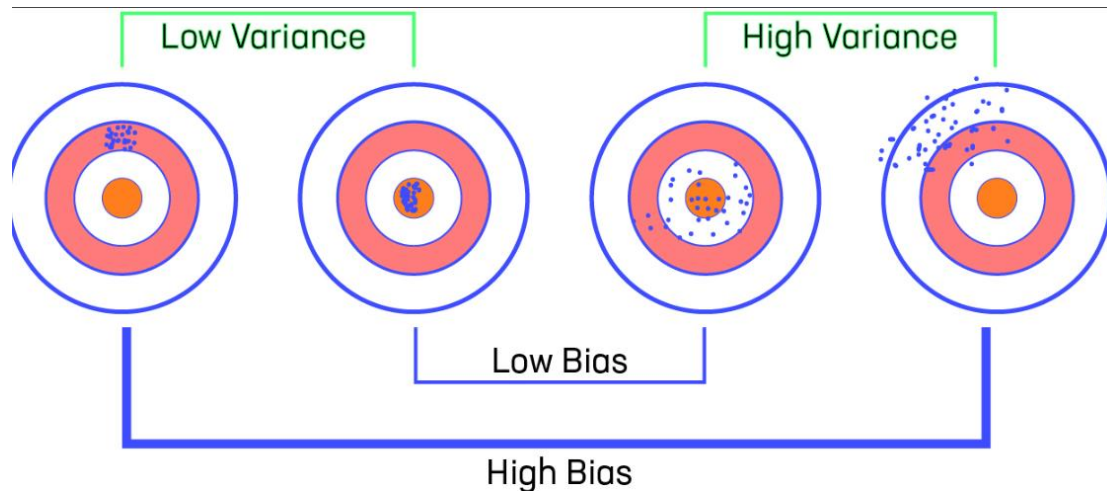
- ✓ **Variance** is the measure of spread in data from its mean position. In machine learning variance is the amount by which the performance of a predictive model changes when it is trained on different subsets of the training data. More specifically, variance is the variability of the model that how much it is sensitive to another subset of the training dataset.
 - i.e. how much it can adjust on the new subset of the training dataset.



High variance → Overfitting
Low variance → Stable predictions
Good ML models balance bias and variance

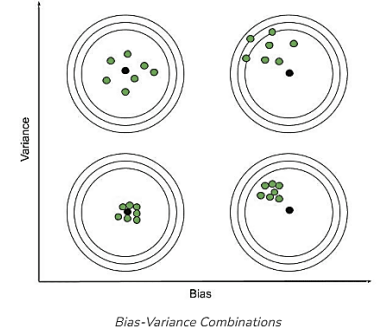
Bias vs Variance

- ✓ **Low Bias:** The average prediction is very close to the target value
- ✓ **High Bias:** The predictions differ too much from the actual value
- ✓ **Low Variance:** The data points are compact and do not vary much from their mean value
- ✓ **High Variance:** Scattered data points with huge variations from the mean value and other data points.
- ✓ To make a good fit, we need to have a correct balance of bias and variance.



Bias-Variance Combinations

Bias vs Variance



- ✓ There can be four combinations between bias and variance.
- ✓ **High Bias, Low Variance:** A model with high bias and low variance is said to be underfitting.
- ✓ **High Variance, Low Bias:** A model with high variance and low bias is said to be overfitting.
- ✓ **High-Bias, High-Variance:** if the **predictions are scattered here and there then that is the symbol of high variance**, also if the **predictions are far from the target then that is the symbol of high bias**.

A model has both high bias and high variance, which means that the model is not able to capture the underlying patterns in the data (high bias) and is also too sensitive to changes in the training data (high variance). As a result, the model will produce inconsistent and inaccurate predictions on average.

Bias vs Variance

- ✓ **Low Bias, Low Variance:** A model that has low bias and low variance means that the model is able to capture the underlying patterns in the data (low bias) and is not too sensitive to changes in the training data (low variance). This is the ideal scenario for a machine learning model, as it is able to generalize well to new, unseen data and produce consistent and accurate predictions. But in practice, it's not possible. So, we trade off between Bias and variance to achieve a balanced bias and variance.
 - **A model with balanced bias and variance is said to have optimal generalization performance.** This means that the model is able to capture the underlying patterns in the data without overfitting or underfitting. The model is likely to be just complex enough to capture the complexity of the data, but not too complex to overfit the training data. This can happen when the model has been carefully tuned to achieve a good balance between bias and variance, by adjusting the hyperparameters and selecting an appropriate model architecture.
- ✓ Sometimes we need to choose between low variance and low bias. There is an approach that prefers some bias over high variance, this approach is called **Regularization**. It works well for most of the classification/regression problems.

What is Regularization?

- ✓ Regularization is a technique to **prevent the model from overfitting by adding extra information** to it.
- ✓ Sometimes the machine learning **model performs well with the training data but does not perform well with the test data**. It means the model is not able to **predict the output when deals with unseen data**, and hence the model is called overfitted. This problem can be deal with the help of a regularization technique.
- ✓ It mainly **regularizes or reduces the coefficient of features toward zero**. In simple words, "In regularization technique, **we reduce the magnitude of the features by keeping the same number of features.**"

Student	Study Hours	Income
A	2	20000
B	4	40000
C	6	60000

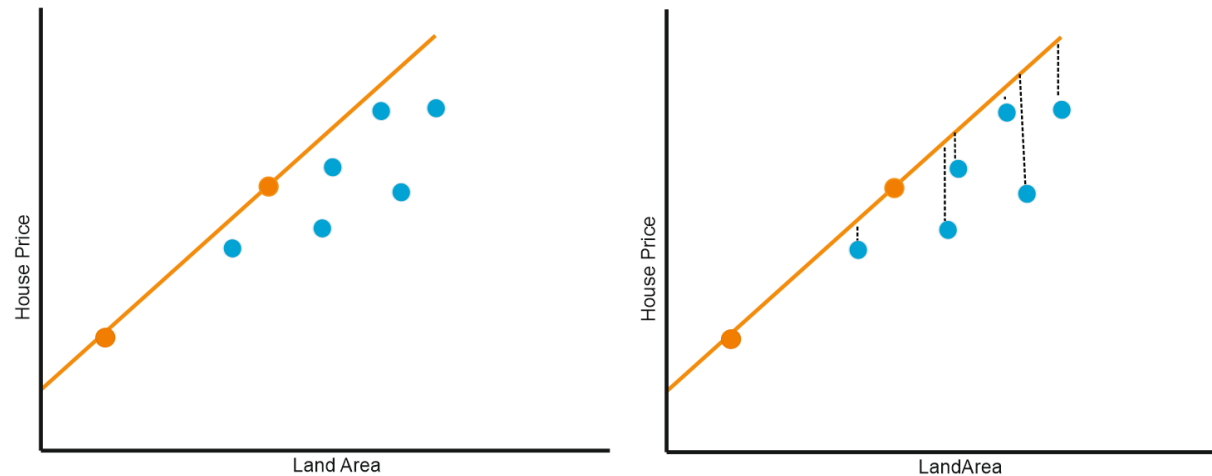


Student	Study Hours	Income
A	0	0
B	0.5	0.5
C	1	1

Min-Max Normalization

Overfitting

- ✓ if we want to predict house prices based on land area, but have only 2 data for training. **Model performs well with these training data (orange dot)**
- ✓ The problem is **Model does not perform well with the test data (blue dot)** and its distribution is like the blue dot below



- ✓ Figure above shows that if we have a regression line in orange, and want to predict test data like the blue dot above, the estimator has a very large variance even though it has a small bias value, and The regression model formed is very good for the train data but bad for the test data. When the phenomenon occurs, the model built is subject to overfitting problems.

Why regularization?

- ✓ Regularization is often used as a solution to the **overfitting** problem in Machine Learning. **Overfitting occurs when a model is too complex and learns noise in the training data.** When a model gets trained with so much data, it starts learning from the noise and inaccurate data entries in our data set. Underfitting occurs when a model is too simple to capture underlying data patterns. Regularization provides a means to find the optimal balance between these two extremes.
- ✓ The primary goal of regularization is to reduce the model's complexity to make it more generalizable to new data, thus improving its performance on unseen datasets.

Techniques of Regularization

- ✓ There are mainly two types of regularization techniques, which are given below:
- ✓ **Ridge Regression** $\Rightarrow$
- ✓ **Lasso Regression** $\Rightarrow$